Supervised Learning: Decision Trees and Bayesian Classifiers

Main instructor: Dr. Chao Huang

Department of Computer Science
Montclair State University

Recall

- What is supervised learning?
  - Labels of training data are provided in advance
- What is classification?
  - predicts categorical class labels
  - classifies data (constructs a model) based on training set and the values (class labels) in a classifying attribute and uses it in classifying new data
  - Typical applications
    - Credit approval
    - Target marketing
    - Medical diagnosis
    - Fraud detection

Recall

- What is supervised learning?
  - Labels of training data are provided in advance
- What is classification?
  - predicts categorical class labels
  - classifies data (constructs a model) based on training set and the values (class labels) in a classifying attribute and uses it in classifying new data
  - Typical applications
    - Credit approval
    - Target marketing
    - Medical diagnosis
    - Fraud detection

Recall

- What is supervised learning?
  - Labels of training data are provided in advance
- What is classification?
  - predicts categorical class labels
  - classifies data (constructs a model) based on training set and the values (class labels) in a classifying attribute and uses it in classifying new data
  - Typical applications
    - Credit approval
    - Target marketing
    - Medical diagnosis
    - Fraud detection

Recall

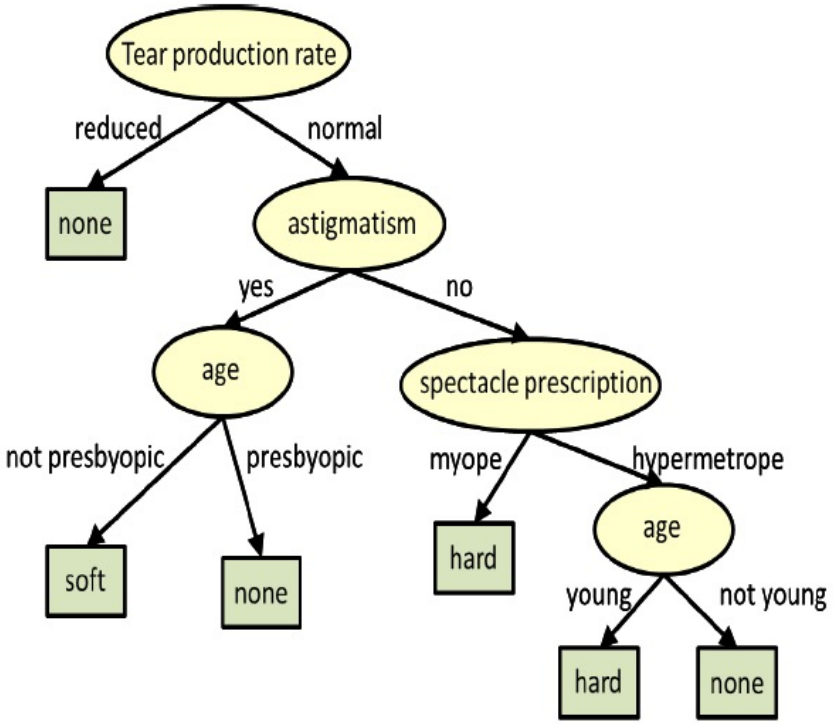
- What is supervised learning?
  - Labels of training data are provided in advance
- What is classification?
  - predicts categorical class labels
  - classifies data (constructs a model) based on training set and the values (class labels) in a classifying attribute and uses it in classifying new data
  - Typical applications
    - Credit approval
    - Target marketing
    - Medical diagnosis
    - Fraud detection

Outline

- Decision trees
- Bayesian classifiers

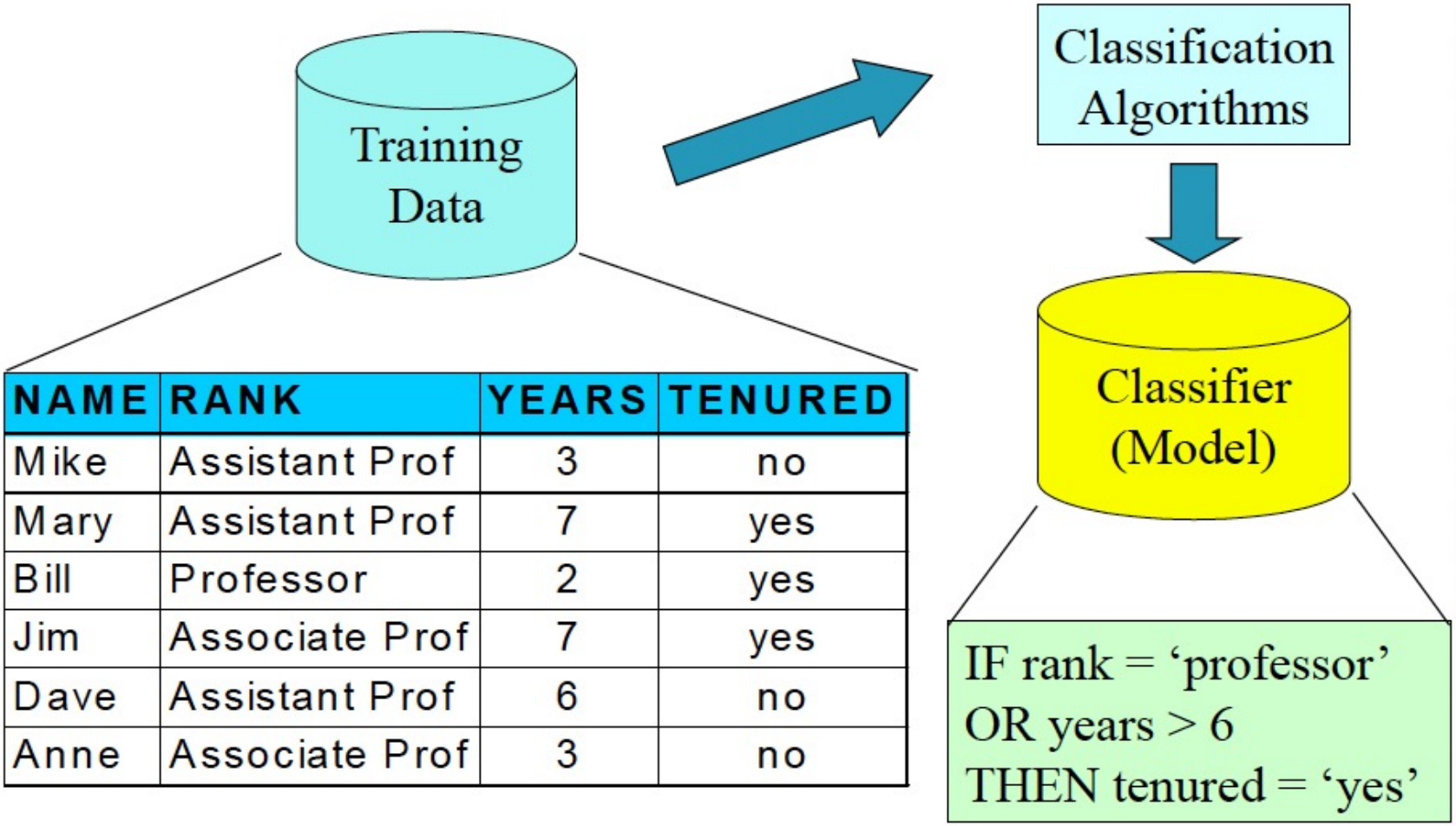
Decision Trees

- Decision trees are decision support tools with tree-like models of decisions and possible consequences
- Each internal node tests an attribute
- Each branch corresponds to an attribute value
- Each leaf node assigns a classification
- In ML, they simulate a human making decisions in a step-by-step manner



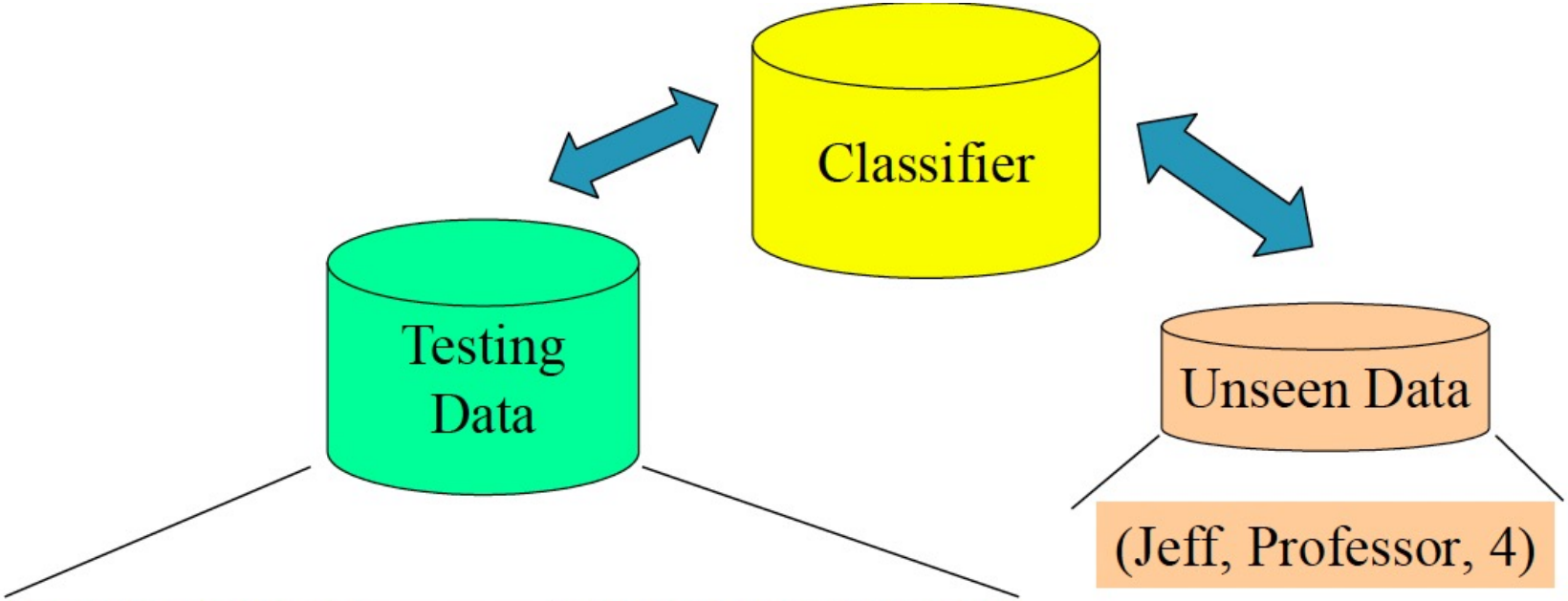
Example: Tree for contact lens prescription
Source: WEKA

Process (1): Model Construction



NAME	RANK	YEARS	TENURED
Mike	Assistant Prof	3	no
Mary	Assistant Prof	7	yes
Bill	Professor	2	yes
Jim	Associate Prof	7	yes
Dave	Assistant Prof	6	no
Anne	Associate Prof	3	no

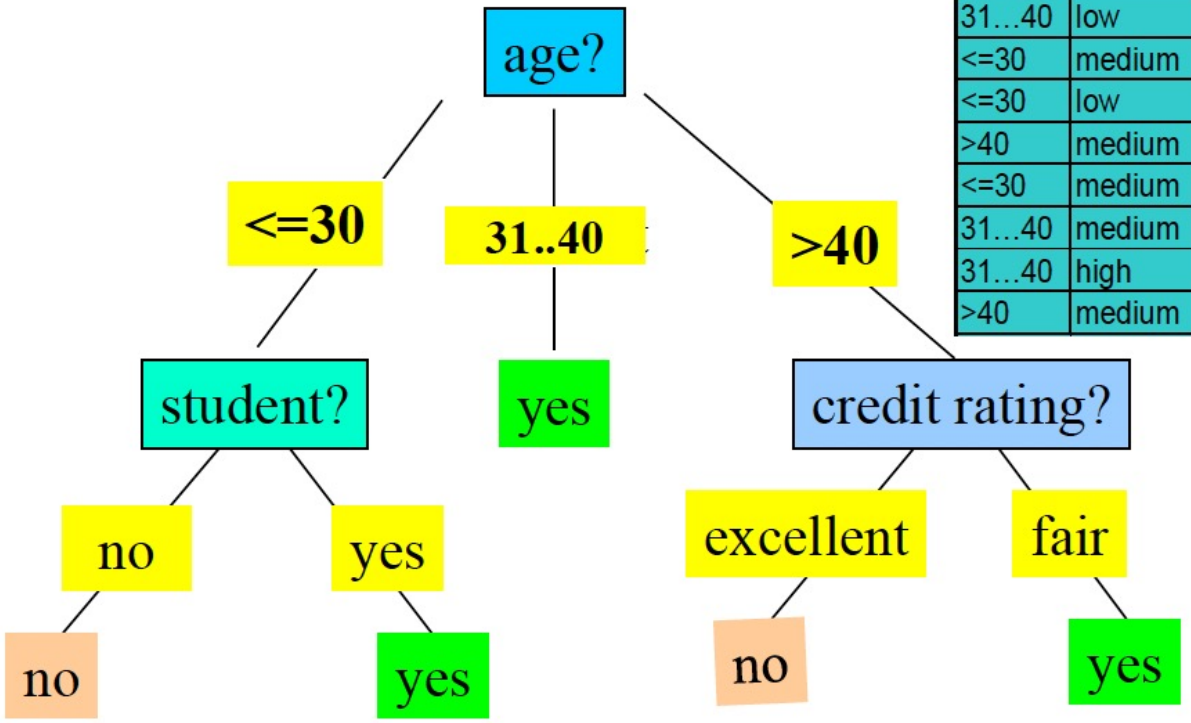
Process (2): Model Prediction



NAME	RANK	YEARS	TENURED
Tom	Assistant Prof	2	no
Merlisa	Associate Prof	7	no
George	Professor	5	yes
Joseph	Assistant Prof	7	yes

Decision Tree Induction: An Example

- Training data set: Buys_computer
- Resulting tree:



age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

How to Train a Decision Tree

- A greedy algorithm
  - Tree is constructed in a top-down recursive divide-and-conquer manner
  - At start, all training samples are at the root
  - Attributes are categorical
  - Examples are partitioned recursively based on selected attributes
  - Test attributes are selected on the basis of a heuristic or statistical measure (e.g., information gain)
- Conditions for stopping partitioning
  - All samples for a given node belong to the same class
  - No remaining attributes for further partitioning - majority voting is used for classifying the leaf
  - No samples left
- Q: how to calculate information gain? $\rightarrow$ using entropy

How to Train a Decision Tree

- A greedy algorithm
  - Tree is constructed in a top-down recursive divide-and-conquer manner
  - At start, all training samples are at the root
  - Attributes are categorical
  - Examples are partitioned recursively based on selected attributes
  - Test attributes are selected on the basis of a heuristic or statistical measure (e.g., information gain)
- Conditions for stopping partitioning
  - All samples for a given node belong to the same class
  - No remaining attributes for further partitioning - majority voting is used for classifying the leaf
  - No samples left
- Q: how to calculate information gain? $\rightarrow$ using entropy

How to Train a Decision Tree

- A greedy algorithm
  - Tree is constructed in a top-down recursive divide-and-conquer manner
  - At start, all training samples are at the root
  - Attributes are categorical
  - Examples are partitioned recursively based on selected attributes
  - Test attributes are selected on the basis of a heuristic or statistical measure (e.g., information gain)
- Conditions for stopping partitioning
  - All samples for a given node belong to the same class
  - No remaining attributes for further partitioning - majority voting is used for classifying the leaf
  - No samples left
- Q: how to calculate information gain? $\rightarrow$ using entropy

Entropy

- What is entropy?
  - A measure of uncertainty associated with a random variable
  - Calculation: for a discrete random variable Y taking m distinct values $\{y_1, \cdots, y_m\}$

$$H(Y) = -\sum_{i=1}^{m} p_i \log(p_i), \quad \text{where } p_i = P(Y = y_i)$$

  - Interpretation:
    - ★ Higher entropy $\rightarrow$ higher uncertainty
- Conditional entropy

$$H(Y|X) = \sum_{x} p(x) H(Y|X = x)$$

Attribute Selection: Information Gain

- Select the attribute with the highest information gain
- p_i: probability that an arbitrary tuple in D belongs to class C_i
- Expected information (entropy) need to classify a tuple in D:

$$Info(D) = -\sum_{i=1}^{m} p_i \log(p_i)$$

- Information needed (after using A to split D into v partitions) to classify D:

$$Info_A(D) = \sum_{j=1}^{v} \frac{|D_j|}{|D|} \times Info(D_j)$$

- Information gained by branching on attribute A:

$$Gain(A) = Info(D) - Info_A(D)$$

Attribute Selection: Example

- Class P: buy_computer = "Yes"
- Class N: buy_computer = "No"
- Aim: select attribute (i.e., age, income, student, credit_rating) with the highest information gain

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

Information Gain for Age

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

- $Info(D) = I(9,5) \triangleq -\frac{9}{14}\log(\frac{9}{14}) - \frac{5}{14}\log(\frac{5}{14}) = 0.940$
  - Out of 14 samples, 9 purchased and 5 did not
- $Info_{age}(D) = \frac{5}{14}I(2,3) + \frac{4}{14}I(4,0) + \frac{5}{14}I(3,2) = 0.694$
  - E.g., $\frac{5}{14}I(2,3)$ means "age$\leq$ 30" has 5 out of 14 samples, 2 yes, 3 no
- Information gain: $Gain(age) = Info(D) - Info_{age}(D) = 0.246$

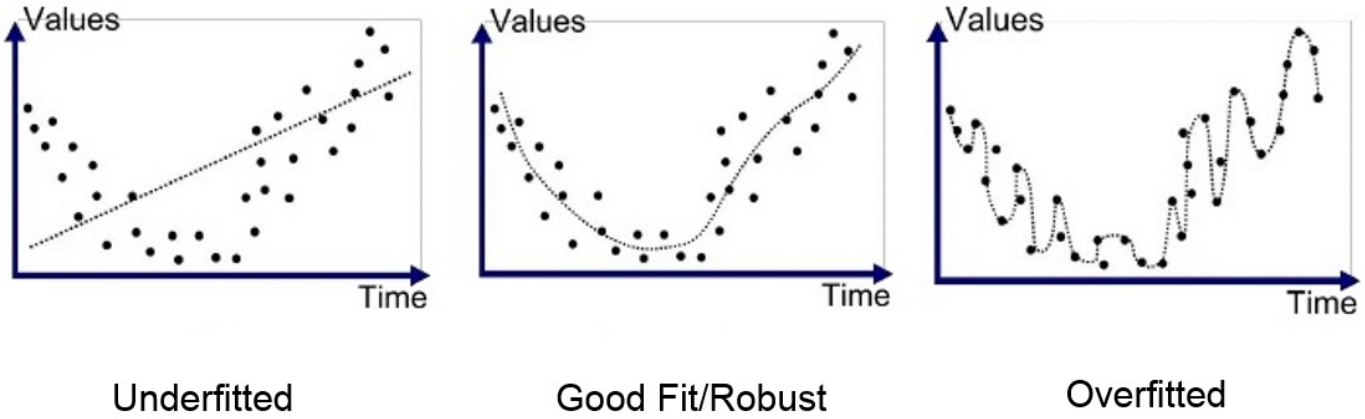
Information Gain

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

- Similarly, (can you calculate the following?)
  - $Gain(income) = 0.029$
  - $Gain(student) = 0.151$
  - $Gain(credit_rating) = 0.048$
- Hence, we should choose age as the attribute

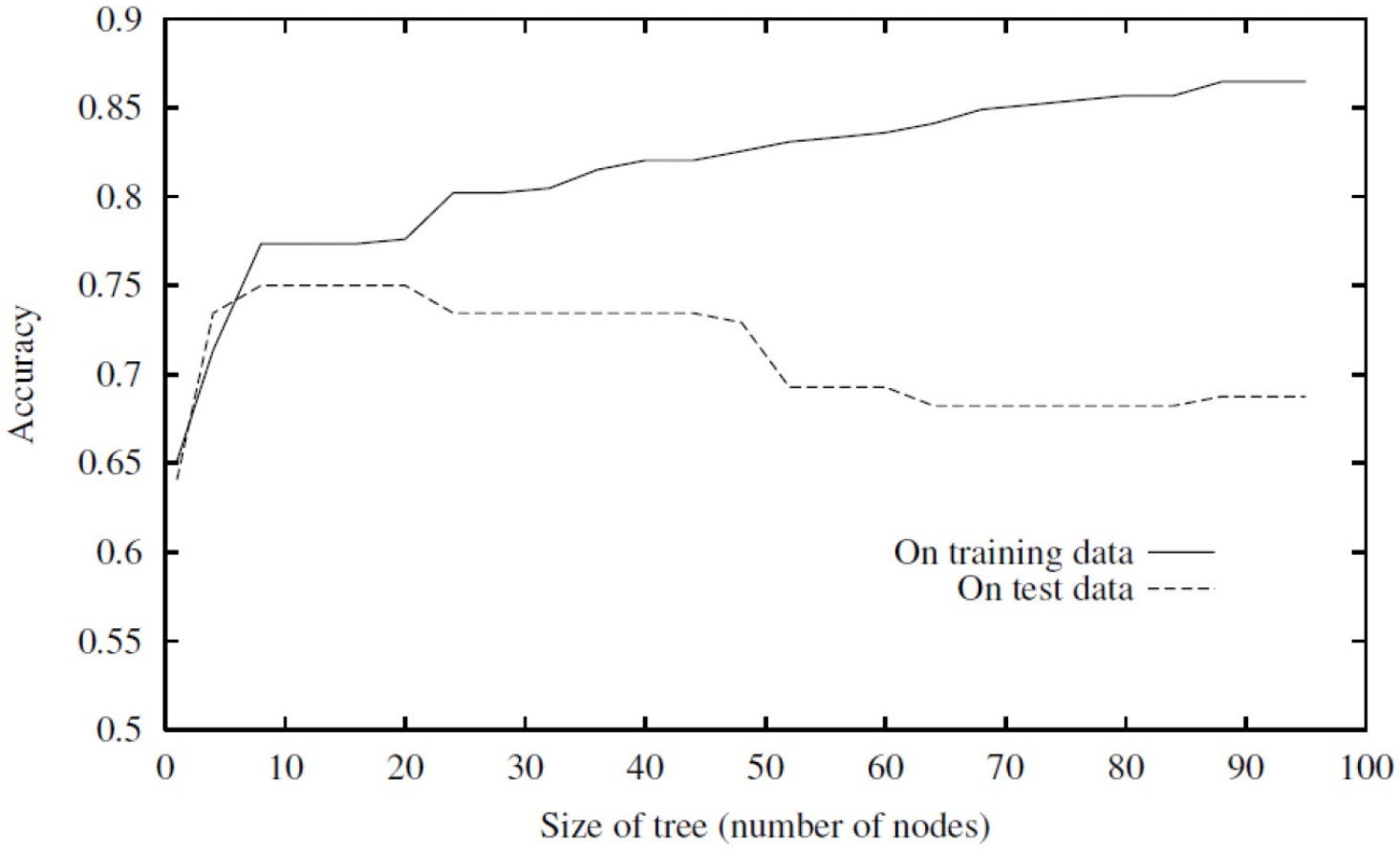
Overfitting

- Decision trees tend to overfit
  - What is overfitting?
  - To be concise, ovefitting occurs when the learned model is too specific to the training set, and hence cannot generalize well to unseen data



- Decision trees may overfit if there are too many branches, some of which reflect anomalies due to noise/outliers

Conduct Tree Pruning Using Accuracy



Pre-pruning and Post-pruning

- Two approaches to avoid overfitting
  - Pre-pruning: halt tree construction early
    - Do not split a node if this leads to a measure (e.g., information gain) falling below a threshold
    - Difficult to choose an appropriate threshold
  - Post-pruning: remove branches from a "fully grown" tree
    - Get a sequence of progressively pruned trees
    - Use a set of data different from the training to decide which is the "best pruned tree"

Bayesian Classifiers

- A statistical classifier: performs *probabilistic prediction, i.e.,* predicts class membership probabilities
- Foundation: Based on Bayes' Theorem.
- Performance: A simple Bayesian classifier, *naïve Bayesian classifier*, has comparable performance with decision tree and selected neural network classifiers
- Incremental: Each training example can incrementally increase/decrease the probability that a hypothesis is correct — prior knowledge can be combined with observed data
- Standard: Even when Bayesian methods are computationally intractable, they can provide a standard of optimal decision making against which other methods can be measured

Bayes' Theorem

- Bayes' Theorem: $P(H|\mathbf{X}) = \frac{P(\mathbf{X}|H)P(H)}{P(\mathbf{X})} = P(\mathbf{X}|H) \times P(H) / P(\mathbf{X})$
  - Let **X** be a data sample ("*evidence*"): class label is unknown
  - Let H be a *hypothesis* that X belongs to class C
  - Classification is to determine P(H|**X**), (i.e., *posteriori probability):* the probability that the hypothesis holds given the observed data sample **X**
  - P(H) (*prior probability*): the initial probability
    - E.g., **X** will buy computer, regardless of age, income, …
  - P(**X**): probability that sample data is observed
  - P(**X**|H) (likelihood): the probability of observing the sample **X**, given that the hypothesis holds
    - E.g., Given that **X** will buy computer, the prob. that X is 31..40, medium income

Naive Bayes Classifier

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

- Class:
  - C1: buys_computer="yes"
  - C2: buys_computer="no"
- Data to be classified:
  - X = (age$\leq$ 30, income=medium, Student=yes, Credit_rating=Fair)?

Naive Bayes Classifier: Example

age	income	student	credit_rating	comp
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

- $P(C_i)$: P(buys_computer = "yes") = 9/14 = 0.643
  P(buys_computer = "no") = 5/14= 0.357
- Compute $P(X|C_i)$ for each class
  P(age = "<=30" | buys_computer = "yes") = 2/9 = 0.222
  P(age = "<= 30" | buys_computer = "no") = 3/5 = 0.6
  P(income = "medium" | buys_computer = "yes") = 4/9 = 0.444
  P(income = "medium" | buys_computer = "no") = 2/5 = 0.4
  P(student = "yes" | buys_computer = "yes) = 6/9 = 0.667
  P(student = "yes" | buys_computer = "no") = 1/5 = 0.2
  P(credit_rating = "fair" | buys_computer = "yes") = 6/9 = 0.667
  P(credit_rating = "fair" | buys_computer = "no") = 2/5 = 0.4
- **X = (age <= 30 , income = medium, student = yes, credit_rating = fair)**

$P(X|C_i)$: P(X|buys_computer = "yes") = 0.222 x 0.444 x 0.667 x 0.667 = 0.044
P(X|buys_computer = "no") = 0.6 x 0.4 x 0.2 x 0.4 = 0.019

$P(X|C_i)*P(C_i)$: P(X|buys_computer = "yes") * P(buys_computer = "yes") = 0.028
P(X|buys_computer = "no") * P(buys_computer = "no") = 0.007

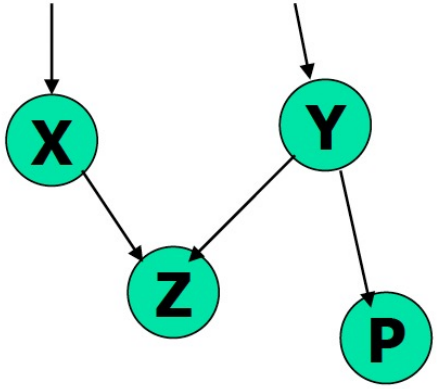
Therefore, X belongs to class ("buys_computer = yes")

Naive Bayes Classifier: Discussion

- Advantages
  - Easy to implement
  - Good results obtained in most of the cases
- Disadvantages
  - Assumption: class conditional independence, therefore loss of accuracy
  - In practice dependencies exist among variables
    - ★ E.g., E.g., hospitals: patients: Profile: age, family history, etc. Symptoms: fever, cough etc., Disease: lung cancer, diabetes, etc.
    - ★ Dependencies cannot be modeled by Naive Bayes
    - ★ What to do? $\rightarrow$ Bayesian belief network

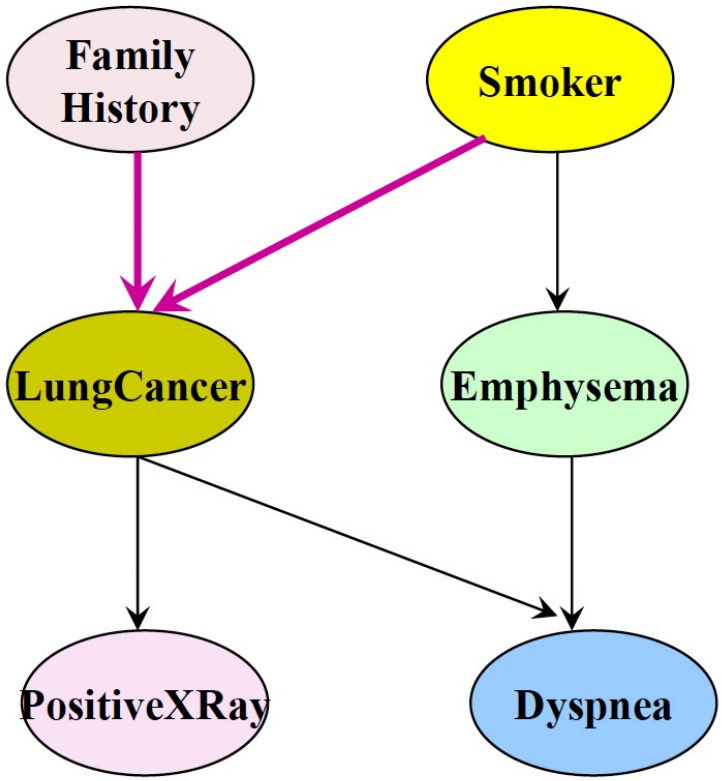
Bayesian Belief Network

- Bayesian belief network allows a *subset* of the variables conditionally independent
- A graphical model of causal relationships
  - Represents <u>dependency</u> among the variables
  - Gives a specification of joint probability distribution



- ❑ Nodes: random variables
- ❑ Links: dependency
- ❑ X and Y are the parents of Z, and Y is the parent of P
- ❑ No dependency between Z and P
- ❑ Has no loops or cycles

Bayesian Belief Network: An Example



Bayesian Belief Networks

The **conditional probability table** (**CPT**) for variable LungCancer:

	(FH, S)	(FH, ~S)	(~FH, S)	(~FH, ~S)
LC	0.8	0.5	0.7	0.1
~LC	0.2	0.5	0.3	0.9

CPT shows the conditional probability for each possible combination of its parents

Derivation of the probability of a particular combination of values of **X**, from CPT:

$$P(x_1,\ldots,x_n) = \prod_{i=1}^{n} P(x_i \mid Parents(Y_i))$$

Summary

- Decision trees
  - Attribute selection using information gain
  - Pruning
- Naive Bayes
- Bayesian belief networks

After-Class Reading

- M¨uller A C, Guido S. Introduction to machine learning with Python: a guide for data scientists. " O'Reilly Media, Inc.", 2016.
  - Sec. 2.1
  - Sec. 2.2
  - Sec. 2.3.1
  - Sec. 2.3.4
  - Sec. 2.3.5

Implementation on Jupyter Notebook

- Decision Trees
- Naive Bayes